RUREX · DESIGN NOTE

UCS to UTF-8

Why every Python string is re-encoded before the regex engine can see it

The engine matches UTF-8. CPython stores `str` as UCS-1/2/4. The conversion in between is the whole problem.

One string, two representations

Python `str` — PEP 393 UCS

- UCS-1 — 1 byte per unit. ASCII and Latin-1, and **the bytes are valid UTF-8 already**.
- UCS-2 — 2 bytes per unit. Up to U+FFFF, and never a valid surrogate pair.
- UCS-4 — 4 bytes per unit. Above U+FFFF; lone surrogates and out-of-range values possible.

The kind follows the string's maximum code point, not its content.

Rust `regex` — UTF-8 bytes

`61` `C3` `A9` `CE` `B2` `F0` `9F` `99`

a é β 🙂 — 1, 2, 2 and 4 bytes

The engine accepts one input format, and cannot be taught another without forking it.

`re` matches code points directly, so it never converts. rurex must produce UTF-8 first, inside the call, with the GIL held.

Encoding is neither free nor small

0.45 GB/s

`PyUnicode_AsUTF8AndSize`, CPython's scalar encoder, cached in the object

0.48 GB/s

Rust's scalar encoder; `String::from_utf16` lands in the same class

x3.0

rurex against `re` on cold non-ASCII strings, before this work

Telegram corpus, 1M short non-ASCII posts, cold strings, GIL, one thread:

1.58 s of work, about 1.0 s of it encoding. The conversion dominated the call, not the matching.

THE FIX

Three ways to view a str as UTF-8

1 **Compact ASCII** — zero copies

The UCS-1 data is a UTF-8 string already. Borrow the object's own buffer.

2 **CPython's cache is warm** — zero copies

An earlier C-API consumer encoded it. Borrow the cached buffer.

3 **Cold non-ASCII** — **1.4 – 3.8 GB/s**

Prefilter the raw UCS storage, then transcode into a reused scratch buffer.

rurex never writes CPython's cache: the buffer is `PyMem_Malloc`-owned, and only `PyUnicode_AsUTF8AndSize` may fill it. Strings warmed solely by rurex re-transcode next scan, traded deliberately against a side cache that would have to survive object identity and lifetime.

PATH 3A

Cheapest answer first: do not transcode at all

When every possible match must begin with one of at most three distinct ASCII bytes, that byte is searched in the raw UCS storage instead.

The literal set is extracted once, at compile time, from `regex-syntax`'s prefix-literal analysis. The haystack is not touched until the pattern exists.

THE CHECK, IN ORDER

1. UCS-1: a plain `memchr` over the units.
2. UCS-2 / UCS-4: a unit-aligned scan, so a hit cannot be a fragment of a neighbouring unit.
3. Byte absent: no match can exist, return the no-match answer.

Cold path only — never for ASCII, a warm cache, or an unvalidated layout. A false positive costs one wasted transcode; a false negative would be a wrong answer, so the byte set must cover every match.

PATH 3B

The transcoder

- Hand-written AVX2 + BMI2 kernel, runtime detection, one `target_feature` function.
- One kernel for all three kinds: units widen to eight `u32` lanes, and no surrogate-pair decoding is ever needed.
- Uniform chunk: a fixed shuffle emits the bytes. Mixed chunk: four candidate bytes staged per lane, compacted by a `pshufb` from a 256-row table.
- The table beats `pext/pdep` control synthesis by 1.6 – 2x.

GB/s OF UTF-8 OUT

mixed Arabic	**1.9**
pure CJK	**3.8**
emoji	**1.5**
scalar fallback	0.48

No new dependency: `encoding_rs` stays scalar on stable Rust, `xutf` is nightly and permissive, `simdutf` would drag C++ into every wheel.

Errors must match CPython exactly

A Python `str` may legally hold a lone surrogate, and a UCS-4 string may hold a value above U+10FFFF. Neither survives an encode.

Both raise `UnicodeEncodeError` at the **first** offending unit: the same error, at the same index, that `PyUnicode_AsUTF8AndSize` reports.

HOW THE KERNEL KEEPS THAT

1. Surrogate and out-of-range lanes detected in bulk, before any SIMD work.
2. One bad lane sends the whole 8-unit chunk to the scalar encoder.
3. Scalar raises at the first bad unit, so the reported index is identical.

Guarded by differential tests against CPython's own encoder: a torture corpus, seeded fuzzing over raw code points including lone surrogates, and every chunk boundary and uniform shape.

THE TRAP

PEP 393 field offsets are not portable

- Reading `kind`, `compact`, `ascii`, the length and the data pointer means reading CPython's fields through raw pointers.
- `PyObject` is 16 bytes on a GIL build, 32 on a free-threaded one.
- `PyCompactUnicodeObject` carried an extra `wstr_length` before 3.12.
- Free-threaded builds widen `state.interned`, shifting the `kind`/`compact`/`ascii` bits upward.

WHAT THE OLD CONSTANTS DID

On a free-threaded build they pointed at the refcount: the ASCII bit was read from a refcount word, and on a differently aligned build the peek could have returned a bogus slice.

Detect once per process, validate against CPython's own behaviour. If it fails, every `str` takes `to_cow`.

Slow on an unknown layout. Never wrong on one.

SUMMARY

One table, in the order a haystack meets them

PATH	CONDITION	COST
Borrow UCS-1	compact ASCII	zero
Borrow cache	non-ASCII, CPython cache warm	zero
Prefilter miss	required ASCII byte absent from UCS	a scan only
Transcode	cold, required byte present	1.4 – 3.8 GB/s
`to_cow`	layout unvalidated	0.45 GB/s

The optimisation is not faster encoding. It is arranging, four times over, that no encoding happens.